

初めての ウェブサイト

初めてのウェブサイト



I. はじめてのウェブサイト

1. はじめに

ウェブ開発の世界へようこそ

みなさん、こんにちは！ ウェブサイトを作ってみたいと思ったことはありますか？ 今の社会では、自分の考えや情報を世界に伝えるために、ウェブサイトを作る技術はとても大きな力になります。

「ウェブ開発はむずかしそう……」と感じるかもしれません。確かに、Facebook（フェイスブック）のような複雑なサイトを、いきなり一人で作るのは難しいです。しかし、安心してください。自分だけのシンプルなウェブサイトなら、少しずつ進むことで、誰でも作ることができます。

このガイドでは、ウェブサイトがどのように動いているのかを、やさしく解説します。自分で情報を伝える力を身につけることは、将来の仕事や自分を表現するために、必ず素晴らしいプラスになるはずです。

準備はいいですか？ まずは、スタートするために必要なものを確認しましょう。

[注意] この資料はMDNの [「Your first website」](#) をやさしい日本語にしたものです。

2. 準備するもの^{じゅんび}

スタートラインに立つ^た

実際にコード（コンピュータへの命令）を書き始める前に、自分のコンピュータで作業ができる準備をすることが大切です。これを「環境構築」と呼びます。

まず、以下の3つのことがスムーズにできるか確認してください。

1. OS（オペレーティングシステム）の操作： WindowsやmacOS^{そうさ}
などの基本的な使い方がわかる。
2. ファイルシステムの理解： ファイルを保存したり、フォルダを作ったり整理したりできる。
3. ブラウザの利用： インターネットで検索をしたり、サイトを見たりできる。

[注意] 「2. ファイルシステムの理解」に自信のない人は、補足資料集の「ファイルとフォルダの基本」を見てください

次に、以下の「道具」を用意しましょう。

- コードエディター： プログラムのコードを書くための専用ソフトです。Visual Studio Codeを使います。すでに皆さんのPCにインストールされています。
- ウェブブラウザ： 主にGoogle Chromeを使います。こちらもインストール済みです。

道具がそろったら、次は「どんなサイトを作るか」という計画を立てるステップです。

3. 計画を立てる

サイトの見た目を決める

いきなりコードを書き始めるのは、設計図なしで家を建てるようなものです。まずは「計画」から始めましょう。最初にしっかり計画を立てることで、作業の途中で迷わなくなり、結果として「情報の整理された見やすいサイト」を作ることができます。

計画を立てるときは、以下のポイントを整理してみてください。

- どんな情報を載せるか：誰に、何を伝えたいですか？
(自己紹介、趣味の紹介など)
- どんなフォントや色を使うか：明るいイメージですか？それともクールで落ち着いたイメージですか？

見た目のイメージが固まったら、いよいよサイトの「中身(文章)」と「形」を作っていく段階に入ります。

[注意] 見た目: 外から見た物事のありさま (例 その車は見た目は美しいが中身は壊れている)

4. サイトの「3つの要素」

HTML、CSS、JavaScript

ウェブサイトは、主に3つの技術が組み合わさってできています。これを「家づくり」に例えて見てみましょう。

技術名	役割（家の例え）	詳しい説明
HTML	構造	段落、リスト、画像など、コンテンツの「意味」や「土台」を作ります。
CSS	見た目（かざり）	色、サイズ、配置、背景など、サイトを「美しく」整えます。
JavaScript	動き	ボタンの反応、アニメーション、ゲームなど「便利な機能」を加えます。

これらの技術は、どれか一つ欠けてもうまくいきません。たとえば、HTMLだけでは文字が並んでいるだけで読みにくく、CSSがないとデザインがバラバラになります。また、JavaScriptがないと動きのない静かなサイトになってしまいます。

これらを正しく組み合わせることで、ユーザーにとって「読みやすく」「使いやすく」「楽しい」ウェブサイトが完成するのです。

[注意] 段落：長い文章を内容などから分けた、言葉の集まり

[注意] リスト：複数の項目や要素を集めたもの

[注意] 画像：が写真しゃしんや絵えなど

5. 世界へ公開する

[注意] 授業では、このステップは実施しません。興味のある方は、[ここをクリック](#)してMDNの関連サイトを参照してください。

パブリッシング

自分のパソコンで作ったファイルは、そのままでは自分にしか見ることができません。これをインターネット上にアップロードして、世界中の誰でも見られる状態にすることを「公開（パブリッシング）」と言います。

公開の手順をシンプルにまとめると、以下ようになります。

1. コードを書き終える：HTML、CSS、JavaScriptを完成させます。
 2. ファイルを整理する：画像やコードのファイルを正しいフォルダにまとめます。
 3. オンラインにアップロードする：「サーバー」と呼ばれる、インターネット上の場所へファイルを送ります。
-

6. まとめと^{つぎ}次のステップ

ここまで、ウェブサイト制作の全体像を見てきました。ウェブ
開発の学^{かい}習^{はつ}は、一度で完璧^{いっぴ}にする必要^{かん}はありません。小さな「でき
た！」を積^つみ重^{かさ}ねていくプロセスそのものが大切^{たいせつ}です。

II. デザインと^{けいかく}計画のガイドブック

ウェブサイトを作るとき、いきなりパソコンでコード（プログラムの言葉）を書き始めるのは、おすすめしません。まずは「計画（プランニング）」をしましょう。

このガイドブックでは、ウェブ^{かいはつ}開発の^{せんもんか}専門家が、^{りゅうがくせい}留学生のみなさんにも^わ分かりやすい「やさしい^{にほんご}日本語」で、サイト^{つく}作りの^{じゅんび}準備について^{おし}教えます。

1. はじめに

なぜ「計画」が大切なのか

家を建てる時に「設計図」が必要なように、ウェブサイトにも計画が必要です。コードを書く前に計画を立てるのには、大きな理由があります。

- 「作り直し」をふせぐ：何も決めずに作ると、あとで「やっぱり違う！」となって、時間をむだにしまいます。
- 最後まで完成できる：最初にやることを決めると、迷わずに最後まで作ることができます。

計画を立てることは、プロのエンジニアも一番大切にしているステップです。

2. ウェブサイトの^{もくてき}目的を^{せいり}整理する

^{さいしょ}最初のプロジェクトでは、^{ないよう}内容を「シンプル」にすることが^{せいこう}成功の^{おお}ヒミツです。^{もくひょう}大きすぎる^た目標を立てると、^{とちゅう}途中で^{いや}嫌になって^{あきら}諦めてしまうからです。

まずは、^{つぎ}次の^{しつもん}3つの^{こた}質問に答えてみましょう。

1. このサイトは^{なに}何についてのサイトですか？（例：好きな^{いぬ}犬について、^{りょこう}旅行したニューヨークについて、パックマン（Pac-Man）についてなど）
2. ^{じょうほう}どんな^{しょうかい}情報を^{なまえ}紹介しますか？（ウェブサイトの^き名前を決めましょう。そして、1つの「^{みだ}見出し（タイトル）」、1つの「^{がそう}画像」、^こ2～3個の「^{だんらく}段落（^{みじか}短い^{ぶんしょう}文章）」を^{かんが}考えてください）
3. ^みどんな^め見た目のサイトにしますか？（^{はいけい}背景は^{なにいろ}何色がいいですか？^{もじ}文字の^{かたち}形は「^{まじめ}まじめ」ですか？それとも「^{かわいい}かわいい」ですか？）」

^{せんもんか}専門家の^{ちえ}知恵：デザインガイド

^{おお}大きなプロジェクトでは、「^{デザインガイド}デザインガイド（^{デザインシステム}デザインシステム）」という^{つく}ルールブックを作ります。例えば、Firefoxには「^たFirefox Acorn Design System」という^{ゆうめい}有名なガイドがあります。これを見ると、^{いろ}プロがどうやって「色」や「^{もじ}文字」のルールを^{どういつ}統一して、^{きれ}きれいに^み見せているかが^わ分かります。

^{もくてき}目的が決まったら、^き次はそれを^え絵に^か描いてみましょう。

[注意] 「プロジェクト」は^{ひと}一つのウェブサイトやアプリケーションを作るための^{けいかく}計画、あるいは、その^{けいかく}計画に^{かか}関わる^{ひとびと}人々の

グループのこ^とです。そう^した計^{けい}画^{かく}のた^めのデジ^たル素^そ材^{ざい}やソ^うス^さコード^さな^どをま^とめ^たもの^を指^さすこ^ともあ^りま^す。

3. デザインのスケッチ

み め つく
見た目のイメージを作る

ペンと紙を使って、サイトの形を自由に描いてみましょう。これを「スケッチ」と言います。

プロも、最初はパソコンを使わず、紙に書くことから始めます。ここで大切なのは、「完璧を目指さないこと」です。有名な画家のゴッホ（Van Gogh）のように上手に描く必要はありません。どこにタイトルがあり、どこに画像があるか、自分が分かれば十分です。

また、ウェブ制作の現場には、2つの役割があります。

デザイナーの しゅるい 種類	しごと ひと どんな仕事をする人？
グラフィックデザイナー	ウェブサイトの 見た目を きれいに デザインする ひと
UXデザイナー	ユーザーが ボタンを 押しやすくしたり、 情報を 見つけやすくしたりする ひと

いま りょうほう しごと じぶん じゅう
今は、あなたがこの両方の仕事をします。自分のアイデアを自由に紙に書いてみてください。

4. テーマカラー（色の名前とコード）を 決める

サイトの印象を決める「色」を選びましょう。

1. 色を選ぶ： カラーピッカーなどのツールで、背景に使いたい色を探します。
2. コードをメモする： 色を選ぶと、#660066 のような「6つの英数字」が表示されます。これを **16進数コード（hex code／ヘックスコード）** と呼びます。

[注意] 16進数コード（hex code）とは コンピュータが「この色です！」と正しく理解するための専用の番号です。あとでコードを書くときに使うので、必ずメモしておきましょう。

5. 画像の準備

ルールを守って探す方法

インターネットにある画像には「持ち主」があります。これを著作権（コピーライト）と言います。勝手に使うと法律のトラブルになることがあるので、注意してください。

「自由に使ってもいい画像」をGoogleで探す方法は、次の通りです。

1. Google画像検索を使う：好きな言葉で検索します。
 2. ツールを使う：検索結果の画面にある「ツール」ボタンをクリックします。
 3. ライセンスを選ぶ：その下に表示される「使用権」をクリックし、「クリエイティブ・コモンズ ライセンス」を選びます。
 4. 保存する：好きな画像を見つけたら、右クリック（MacはCtrl + クリック）をして、「名前を付けて画像を保存」を選びます。
-

6. フォント（文字の形）の選択

フォントには2つのタイプがあります。

- ウェブセーフフォント：どのパソコンにも最初から入っている安心なフォントです（例：Arial, Times New Roman）。
- Google Fonts：デザイン性が高いフォントを無料で借りられるサービスです。

ここでは、Google Fontsを使う手順を説明します。

1. Google Fontsへ行く：サイトのイメージに合うフォントを探します。
2. コードを取得する：好きなフォントを選び、「Get font」ボタンを押してから、「Get embed code（埋め込みコードを取得）」をクリックします。
3. 2つのコードを保存する：画面に表示された2つのコードブロック（2つのまとまったコード）を両方ともコピーして、メモ帳などに保存してください。フォントを動かすには、この2つが両方必要です。

 注意：商業利用について フォントにもライセンスがあります。勉強で使うのは大丈夫ですが、将来、仕事でサイトを作るときは、必ず「商売（ビジネス）で使ってもいいか」を確認しましょう。

7. まとめと次のステップ

つか
お疲れさまでした！これで「サイトの設計図」と「素材」がすべて揃いました。

- 計画：どんな内容にするか決めた。
- スケッチ：レイアウトを紙に書いた。
- 素材：色のコード、画像、フォントの準備ができた。

しっかりとした「土台」ができたので、次のステップである「HTMLやCSSを書く作業」を、自信を持って進めることができます。

準備はバッチリです。さあ、次は実際にコンピュータで、あなただけのウェブサイトをかたちにしていきましょう！

III. ウェブサイトの形を作ろう

ウェブサイトを作るための 最初の一步へ ようこそ！ プログラミングや ウェブ制作を 始めるとき、最初に 覚えるのが「HTML」です。

HTMLは ウェブサイトの「骨組み」、つまり 構造を作るための言葉です。この 基本を 正しく 知ることは、ウェブサイトを作るプロ（仕事をする人）になるために、とても大切なことです。

[注意] 「骨組み」は、もとは「からだの骨の組み立て」のことですが、そこからの「例え」で、建造物・機械などの基礎的な構造の部分のことも指します。

1. HTMLとは何？：ウェブの^{きほん}基本を^{りかい}理解する

HTML (HyperText Markup Language) は、テキスト (文字) に「印」をつけて、ウェブサイトの^{こうぞう}構造を作るための「マークアップ言語」です。

HTMLの^{やくわり}役割を^{たと}わかりやすく^{にもつ}例えると、「^は荷物に^{みだ}ラベルを貼る^{さぎょう}作業」に似ています。ただのテキストに「これは^{みだ}見出しです」「これは^{だんらく}段落です」という^はラベルを貼ることで、ブラウザ (Google Chromeなど) がその^{いみ}意味を^{りかい}理解できるようになります。

- ^{ようそ}要素 / Elementとは：テキストを「タグ」と呼ばれる^よ記号で^{かこ}囲んだセットのことです。
- ブラウザへの^{でんごん}伝言：タグを使うことで、ブラウザに「ここから^{だんらく}ここまでが段落だ」と^{つた}伝えることができます。

HTMLを使う^{つか}理由^{りゆう}：もしHTMLがなければ、すべての^{もじ}文字が^{いちぎょう}一行につながってしまい、とても^よ読みにくくなります。また、^め目が見えにくい人が使う「音声読み上げソフト」も、どこが^{だいじ}大事な^{ばしよ}場所なのかわからなくなってしまうます。HTMLで^{ただ}正しく^{こうぞう}構造を作ることで、^{だれ}誰にでも^{いみ}意味が^{つた}伝わるページになります。

2. HTMLファイルの^{き ほん}基本^{かたち}の形

ウェブページを^{ただ}正しく^{うご}動かすためには、^き決まった^{かた}「型」が^{ひつよう}必要です。

^{しゅよう}主要な^{こうせいようそ}構成要素

MDNのルールに^{もと}基づいた、^{き ほん て き}基本的な^{い か}パーツは^{とお}以下の通りです。

- `<!doctype html>`：^{ただ}ページを^{ひょうじ}正しく表示させるために、ファイルの^{うえ}いちばん^か上に書かなければならないルールです。
- `<html>`：ページの^{ないよう}すべての^{つつ}内容を^{そとがわ}包む「^{はこ}いちばん外側の箱」です。
- `<head>`：^{がめん}画面には^み見えないけれど、^{けんさく}ブラウザや検索エンジン^{たいせつ}のための^{じょうほう}大切な^{じょうほう}情報（^い情報/メタデータ）を^{ばしょ}入れる場所です。
- `<title>`：ブラウザの「タブ」に^で出る^{なまえ}名前や、ブックマーク（^きお気に入り）したときの^{なまえ}名前になります。
- `<body>`：^{じっさい}実際にユーザーが^み見るコンテンツ（^{ぶんしょう}文章や^{がそう}画像など）を^いすべて^{ばしょ}入れる場所です。

^{たいせつ}メタデータ（^{せってい}大切な^{か ち}設定）の価値

`<head>`の^{なか}中には、^{い か}以下の^{せってい}ような^か設定を書きます。

- ^{も じ}文字コード（UTF-8）：^かこれを^{にほんご}書かないと、日本語などが^{へん}変な^{きごう}記号になる「^{も じ ば}文字化け」が^お起きることがあります。
- ^{ビューポート}（viewport）：^{がめん}スマートフォンの画面サイズに^あ合^{ひょうじ}わせて、^{ひつよう}きれいに表示するために必要です。

^{こうそう}タグの^い構造：^こ入れ子

HTMLは「タグの ^{なか}中に ^{べつ}別の ^{はい}タグが ^{はい}入る」という「入れ子」の ^{かたち}形になります。

```
<html> <!--一番外側の箱-->
  <head> <!--ブラウザのための情報の箱-->
    <title> ページの名前 </title>
  </head>
  <body> <!--私たちが見る中身の箱-->
    <h1> タイトル </h1>
    <p> 文章の 段落（だんらく） </p>
  </body>
</html>
```

[注意] 「入れ子」：大きな箱の中に小さな箱が入っている、
というように、あるものの^{なか}中に、よく似たあるもの^{はい}が入っている^{ようす}様子を指す日本語です。

3. テキストに意味をつける：見出し、段落、リスト

文章を分かりやすくするために、適切なタグを使います。

- 見出し（<h1>～<h6>）：本の「タイトル」や「章」のように、情報の優先順位を示します。<h1>がもっとも重要なタイトルで、数字が大きくなるほど小さな見出しになります。
- 段落（<p>）：ふつうの文章をまとめるためのタグです。
- リスト（箇条書き）：
 - （順序なしリスト）：買い物リストのように、順番が関係ないときに使います。
 - （順序ありリスト）：料理の手順のように、順番が大切なときに使います。
 - ※どちらも、中身の項目は タグで囲みます。

HTMLコメント（<!-- -->）のメリット：コードの中に <!-- --> と書くと、ブラウザには表示されません。これは「自分や仲間」のためのメモです。あとで見直したときに、何をしたのかすぐに分かるようになります。

4. 画像を表示する：要素の使い方

画像を表示するには 要素を使います。

- src属性：画像がある場所（パス）を指定します。
- alt属性：画像の説明を文字で書きます。
 - 重要性：目が不自由な人が使うソフトがこの文字を読み上げます。また、画像がエラーで出ないときにも代わりに表示されます。
 - 良い例：alt="Firefoxのロゴ：地球を包む燃えるキツネ" のように、何が書いてあるか具体的に書くのが正しい使い方です。

空要素 / void element： は文章を囲まないで、終わりのタグ（）がありません。これを「空要素」と呼びます。

ファイル管理のアドバイス：画像は images/ という名前のフォルダを作って、その中に入れましょう。HTMLからは `src="images/写真の名前.jpg"` のように指定します。こうすることで、たくさんのファイルをきれいに整いできます。

5. リンクを作^{つく}ってページをつなぐ：<a> ようそ 要素

ウェブ（蜘蛛^{くも}の巣^す：くものす）の名^な前^{まえ}のとお^おり、ページどうしをつなぐの^のが「リンク」です。

リンクの作^{つく}り方^{かた}

1. リンクにしたいテキストを^{えら}選びます。
2. そのテキストを <a> タグで^{かこ}囲みます。
3. href 属性に、行^{ぞく}きたい先^いのアドレス（URL）を^{さき}書^かきます。

```
<a href="https://www.mozilla.org/">Mozillaのウェブサイト</a>
```

大切な^{たいせつ}注意^{ちゅうい}点^{てん}：アドレスを^か書^{かな}くときは、必ず^{かなら} `https://` などの通信^{つうしん}のルール（プロトコル）から^か書^{はじ}き始めてください。これがないと、リンクが^{ただ}正^{ただ}しく動^{うご}きません。また、リンクにする^{ことば}言葉^{ことば}は「ここを^いクリッ^{ことば}ク」ではなく「Mozillaの^{えら}ウェブサイト」のよう^{えら}に、どこに^い行くのか^{ことば}わかる^{えら}言葉^{えら}を^{えら}選びましょ^{えら}う。

6. まとめ：自分のウェブサイトを作ってみよう

「構造 (HTML)」「テキスト」「画像」「リンク」が組み合わさることで、一つのページが完成します。

初心者のための チェックリスト

公開する 前に、これを 確認しましょう：

- ☐ タグの 閉じ忘れは ありませんか？ (以外、終わりのタグが 必要です)
- ☐ ファイルの名前は 正しいですか？ (大文字と 小文字の 間違いは ありませんか？ Index.html と index.html は 違います)
- ☐ 画像の パス (フォルダの場所) は 正しいですか？ (images/ フォルダの 中に 画像が ありますか？)
- ☐ リンクの https:// を 忘れていませんか？
- ☐ alt属性は わかりやすく 書きましたか？

次のステップ：HTMLで 形が できたら、次は 「CSS」 を 学んで 色や デザインを きれいに しましょう。さらに 「JavaScript」 を 使えば、動きのある ページに なります。

IV. ウェブサイトをデザインする CSS

1. はじめに：CSSとは何でしょうか？

ウェブサイトを作るとき、文字や画像などの「内容」を準備するだけでは、まだデザインが完成していません。そこで使われるのが**CSS**（シーエスエス）です。

CSSは、ウェブサイトの「見た目」を整えるための専用の言葉です。HTMLで作成した文章に色をつけたり、大きさを変えたり、好きな場所に並べたりすることができます。

HTML（内容）とCSS（スタイル）の違い

ウェブサイトは、主に2つの役割が組み合わさってできています。

- HTML（内容）：「ここにタイトルがある」「ここに文章がある」という、サイトの骨組み（土台）を作ります。
- CSS（スタイル）：「タイトルを青色にする」「文章の横幅を600ピクセルにする」といった、見た目を美しく飾ります。

CSSを使ってデザインを整えることで、ウェブサイトはもっと読みやすくなり、訪れた人に「使いやすい」と感じてもらえるようになります。

CSSの定義

- スタイルシート言語：プログラミング言語ではなく、見た目を指定するための特別な言葉です。
- 要素の装飾：HTMLで書かれた特定の場所を選んで、見た目を変えることができます。

次のセクションでは、CSSを実際にどうやって書くのか、その基本ルールを学びましょう。

2. CSSの書き方（構文）を覚えましょう

CSSを書くときには、決まった形（構文）があります。「どこの」「何を」「どう変えるか」という順番で指示を書きます。

CSSを構成するパーツ

CSSは、以下のパーツを組み合わせて作ります。

1. セレクター（選びたい場所）：デザインを変えたいHTMLの要素を指定します。
2. プロパティ（変えたい項目）：色、大きさ、場所など。
3. 値（設定する内容）：具体的にどんな色にするか、どれくらいの大きさにするかを決めます。

宣言とルールセット

CSSでは、プロパティと値を組み合わせたペアを「宣言」と呼びます。また、セレクターと { } で囲まれた複数の宣言をまとめたものを「ルールセット（Ruleset）」と呼びます。

書き方のルールとコード例

プロパティの後には必ずコロン : を、値の後には必ずセミコロン ; を書きます。

```
p {  
  color: red; /* 宣言1：文字の色（プロパティ）を赤（値）にします */  
  font-size: 16px; /* 宣言2：文字の大きさ（プロパティ）を16ピクセル（値）にします */  
}
```

書き方を理解したら、次はCSSをHTMLに読み込ませる方法を確認しましょう。

3. HTMLにCSSを読み込ませる方法

CSSを書くだけでは、ウェブサイトにデザインは反映されません。HTMLとCSSを正しくつなぐ手順が必要です。

正確な設定が大切な理由

ウェブサイトを作るときは、ファイルを整いするために styles という名前のフォルダを作り、その中に style.css を作成します。

HTMLファイルの `<head>` (ヘッド：サイトの設定を書く場所) 内に、以下の1行を書きます。

```
<link href="styles/style.css" rel="stylesheet" />
```

【トラブルを避けるポイント】

フォルダ名やファイル名が1文字でも間違っていると、デザインは全く表示されません。

styles/style.css という「道すじ (パス)」が実際のフォルダ構成と合っているか慎重に確認しましょう。

4. 文字のデザインをきれいにする（フォントとテキスト）

文字の見た目は、サイトを訪れる人の「読みやすさ」に大きな影響があります。適切なフォント選びは、サイトを信頼できる印象にするための戦略です。

Google Fonts の利用

標準のフォント以外を使いたいとき、Google Fonts のサービスを使います。HTML の `<head>` に自分のstyle.cssより前に読み込ませます。

```
<!-- Google Fontsの読み込み例（自分のcssより前に書きます） -->
<link href="https://fonts.googleapis.com/css?family=Open+Sans"
<link href="styles/style.css" rel="stylesheet" />
```

読みやすさを高めるプロパティ

- font-family：フォントの種類
 - font-size：文字の大きさ
 - text-align：文を中央に寄せる
 - line-height：行間
 - letter-spacing：文字の間隔
-

5. すべては「箱（ボックス）」でできている

CSS の世界では、すべての要素（見出し・文章・画像など）は「箱」の中にあると考えます。これを **ボックスモデル** と呼びます。

要素	やさしい説明
Content	文字や画像が入る中心部
Padding	中身と枠線の内側の余白
Border	枠線そのもの
Margin	枠線の外側の余白

6. ページ全体のレイアウトを整える（色と配置）

背景に色をつけ、body の幅を600pxにし、中央寄せする典型的な設定です。

```
html {  
  background-color: #00539F; /* ページ全体の背景を青にします */  
}  
  
body {  
  width: 600px; /* 全体の幅を600ピクセルに固定します */  
  margin: 0 auto; /* 上下は0、左右は「自動 (auto)」で分けて中央に寄せます */  
  background-color: #FF9500; /* bodyの背景を赤みのあるオレンジにします */  
  padding: 0 20px 20px 20px; /* 内側に余白を作ります */  
  border: 5px solid black; /* 黒い枠線で囲みます */  
}
```

margin: 0 auto; の仕組み：

ブラウザが左右の空きを半分ずつ割り当て、body が中央に配置されます。

タイトルに影をつける

```
h1 {  
  text-shadow: 3px 3px 1px black; /* 横に3px、縦に3px、ぼかし1px、 */  
}
```

画像を中央に置く

```
img {  
  display: block; /* 画像を「ボックス」として扱うように変えます */
```

```
margin: 0 auto; /* これで中央に寄るようになります */  
}
```

7. おわりに：^{すてき}素敵なウェブサイト^{つく}を作る ために

CSS の^{き そ}基礎は、^{しょうらい}将来のアニメーションや高度レイアウトの^{どだい}土台になります。

1. ^{あたい}値を変えて効果を確認
2. 他サイトを^{はこ}箱として観察
3. 1項目ずつ保存しながら確認

^{たの}楽しみながら、美しいサイトを完成させてください！

V. ウェブサイトを動かす

JavaScript

ウェブサイトを見たときに、ボタンを押して画面が変わったり、
画像が動いたりすることはありませんか？

それは「JavaScript」という言葉が働いているからです。

このガイドでは、プログラミングが初めての人や、日本語を
勉強中の留学生にも分かりやすく、JavaScriptの基本を説明しま
す。

1. はじめに：JavaScriptとは何か^{なに}

ウェブサイトを作るとき、HTMLは「骨組み」を、CSSは「見た目」を作ります。そこに「動き」を加えるのがJavaScriptの役割です。

JavaScriptを使うと、ユーザーの操作に合わせてサイトの表示を変えることができます。

ただ情報を読むだけのページから、ユーザーといっしょに動く「体験」へと変えることができる、とても大切な技術です。

JavaScriptでできること

JavaScriptを使って、以下のような動きをすることができます：

- チェックする： フォームに入力されたデータが正しいか確認します。
 - 動かす： ボタンをクリックしたときに、何かを実行します。
 - 計算する： ゲームの点数や、数字の計算をおこないます。
 - デザインを変える： スタイル（色や大きさ）を自由に変えます。
 - アニメーション： 画像や文字をスムーズに動かします。
-

2. プログラミングの「^{きほん}基本の^{どうぐばこ}道具箱」

プログラムを^{つく}作るとは、いくつかの^{どうぐ}道具を^く組み合わせて^あ「^{めいれい}命令」をつくることです。JavaScriptを^{うご}動かす4つの大切な^{たいせつ}概念が^{がいねん}あります。

1. ^{へんすう}変数 (Variables) : データを一時的^{いちじてき}に入れておく「^い入れ^{もの}物」。
 2. ^{かんすう}関数 (Functions) : いくつかの^{めいれい}命令をまとめた^{べんり}便利なセット。
 3. ^{じょうけんぶんき}条件分岐 (Conditionals) : 「もし～ならA、そうでなければB」。
 4. イベント (Events) : 「クリックされた」などの^{どうさ}動作のきっかけ。
-

3. 実践：ウェブサイト^{じっせん}にJavaScript^いを入れる

HTML、CSS、JavaScript の3つが連携^{れんけい}してページ^{うご}が動きます。

設定^{せってい}の手順^{てじゆん}

1. scripts フォルダを作り、その中に main.js を作成します。
2. HTMLファイルの <head> の終わる直前に次^かを書きます：

```
<script src="scripts/main.js"></script>
```

- CSSの <link> と同じ仕組みで、JavaScript を読み込み^よみます。

要素^{ようそ}を選^{えら}んで動^{うご}かす

```
let myHeading = document.querySelector('h1');  
myHeading.textContent = 'Hello world!';
```

- 1行目： <h1> を選^{えら}び、myHeading に入^いれます。
- 2行目： textContent を “Hello world!” に変^かえます。

ヒント： // ^か ^{ぎょう} で書いた行はコメントとして無視^{む し}されます。

4. 応用：画像チェンジャーと条件分岐

クリックで画像が入れ替わる仕組みです。

手順

1. 画像を JavaScript で選ぶ
2. クリックされるのを待つ
3. 今の src を確認
4. if 文で切り替える

実践コード

```
let myImage = document.querySelector('img');

myImage.onclick = function() {
  let mySrc = myImage.getAttribute('src');
  if (mySrc === 'images/images/photo1.jpg') {
    myImage.setAttribute('src', 'images/photo2.jpg');
  } else {
    myImage.setAttribute('src', 'images/photo1.jpg');
  }
}
```

- myImage.onclick：クリック時のイベント
 - getAttribute：現在の src を調べる
 - setAttribute：src を書き換える
 - if...else：条件で処理を切り替え
-

6. ま と め と つぎ の ステ ッ プ

つか
お疲れさまでした！これでJavaScriptのきそ基礎を学びました。

- HTML/CSS：ページのどだいみめ土台と見た目
- JavaScript：うごぎじゅつ動きを加える技術

JavaScript は ゆっくり学べば確実に身につきます。
MDNの教材（日本語 / 英語）もとても良い参考になります。

少しずつ練習して、ぜひあなたのオリジナルのうご動くウェブサイト
つくを作ってください！

ほじょしりょう 補助資料

「^{はじ}初めてのウェブサイト」を^{まな}学ぶ^{うえ}上で、たすけになる・^{さんこう}参考になる
^{しりょう}資料をまとめました。

ウェブ^{かい}開発^{はつ}のためのファイルとフ ォルダ^{きほん}の基本ガイド

ウェブサイトを作^{つく}るとき、たくさんのファイルを使^{つか}います。このガイドでは、プロのエンジニアが最初^{さいしよ}に覚^{おぼ}える「ファイルの整理^{せいり}・整頓^{せいとん}」のルールを、やさしく説^{せつめい}明します。

1. はじめに：なぜファイルの整理が大切なのか

ウェブサイトは、文字のデータ、プログラム、画像など、多くのファイルで構成されています。これらのファイルは、お互いに呼び出し合って動いています。

もしファイルがバラバラな場所にあると、ウェブサイトは正しく動きません。最初にしっかり整理するルールを決めることが、ウェブサイト作りを成功させる一番の近道です。

次は、自分のパソコンのどこにファイルを作るべきか説明します。

2. ウェブサイトの場所を決めよう

ウェブサイトのファイルは、一つのフォルダにまとめて入れます。
このフォルダの中身は、インターネット上のコンピュータ（サーバー）に公開するときと「同じ形」にする必要があります。これを「ミラー（鏡）」と呼びます。自分のパソコンで動くものは、サーバーでも同じように動かなければいけないからです。

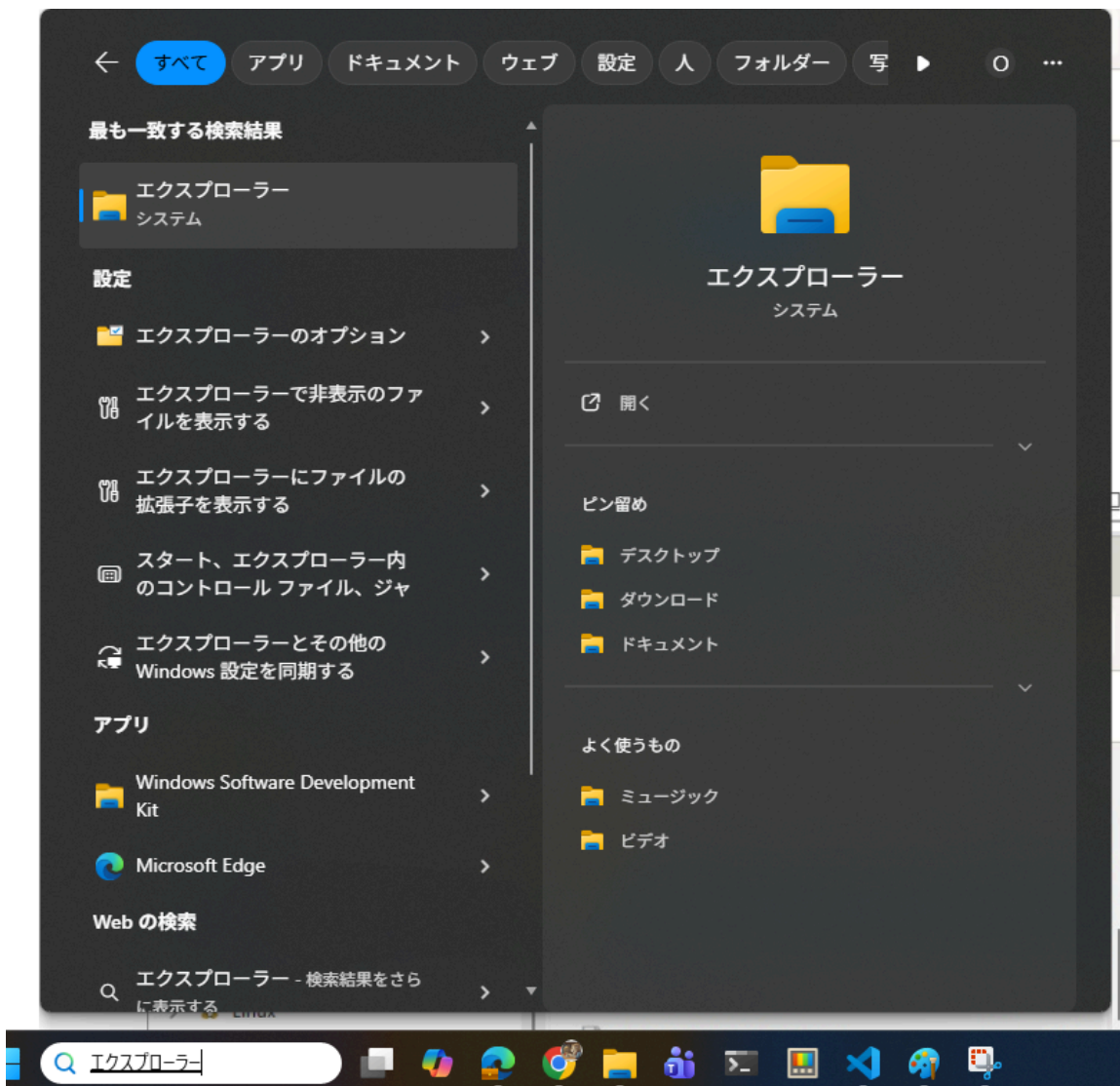
おすすめのフォルダ階層

MDN（ウェブ開発の標準的な資料）では、次のような作り方をすすめています。

(1). まず、すべてのプロジェクトを保存する web-projects というフォルダを作ります。

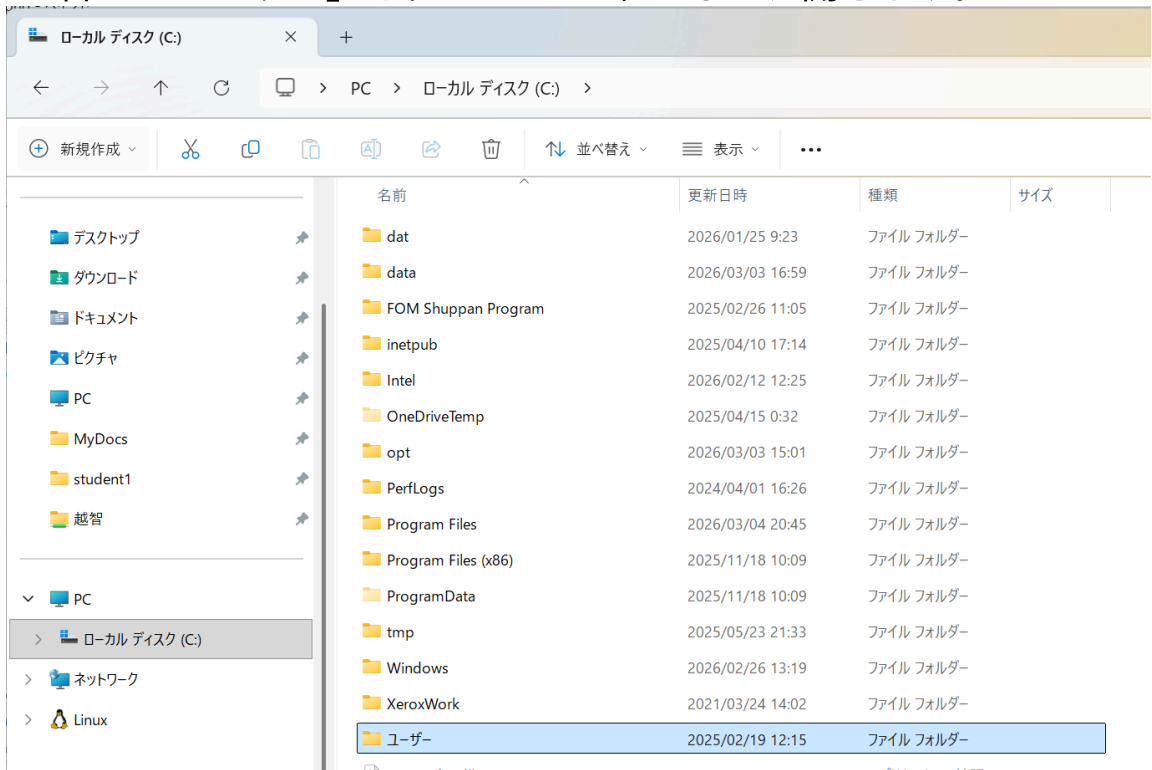
今回は、PC > ローカルディスク(C:) > ユーザー > student1 > source の下につくりましょう。

1. ステータスバーの検索窓に「エクスプローラー」と入力します。

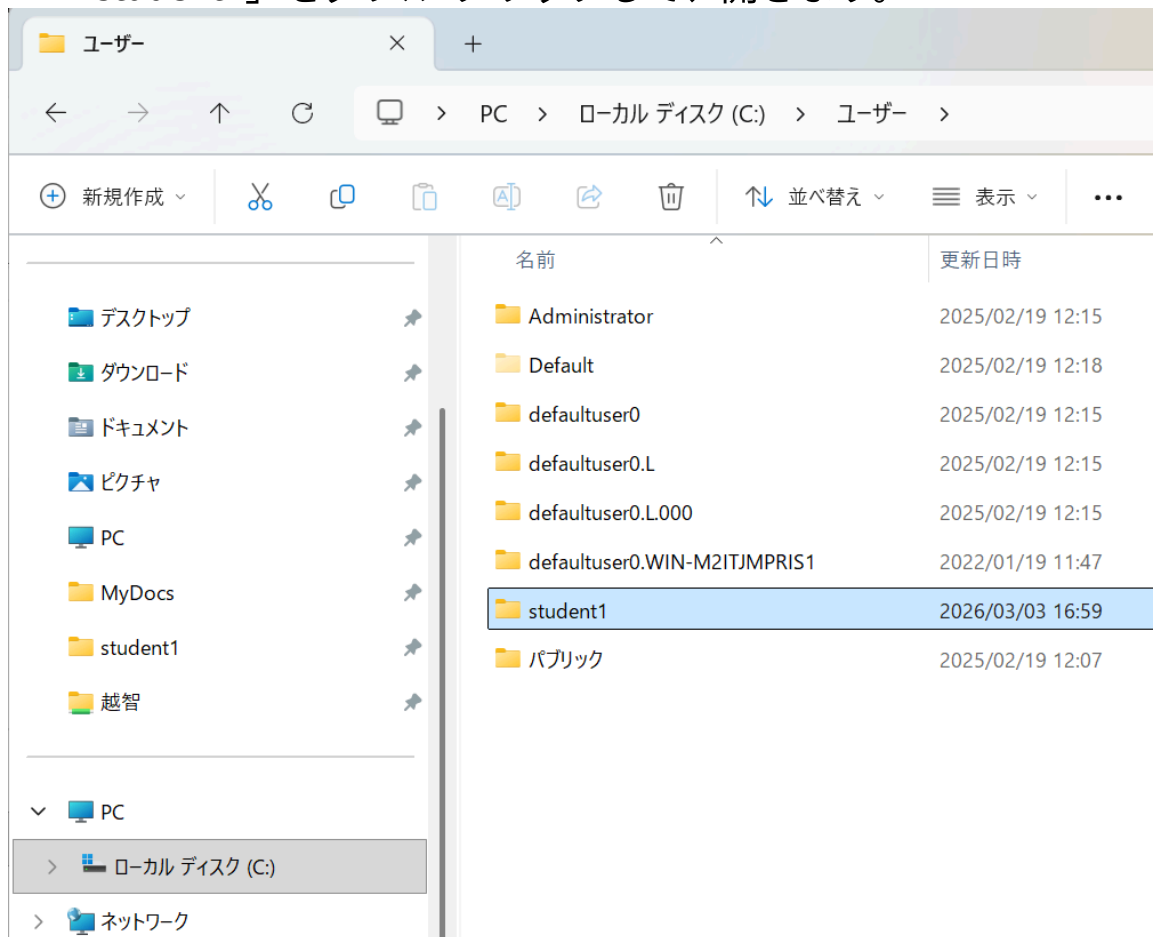


2. ^{ひだり} 左の「PC」の下 ^{した} の「ローカルディスク(C:)」をクリックします。

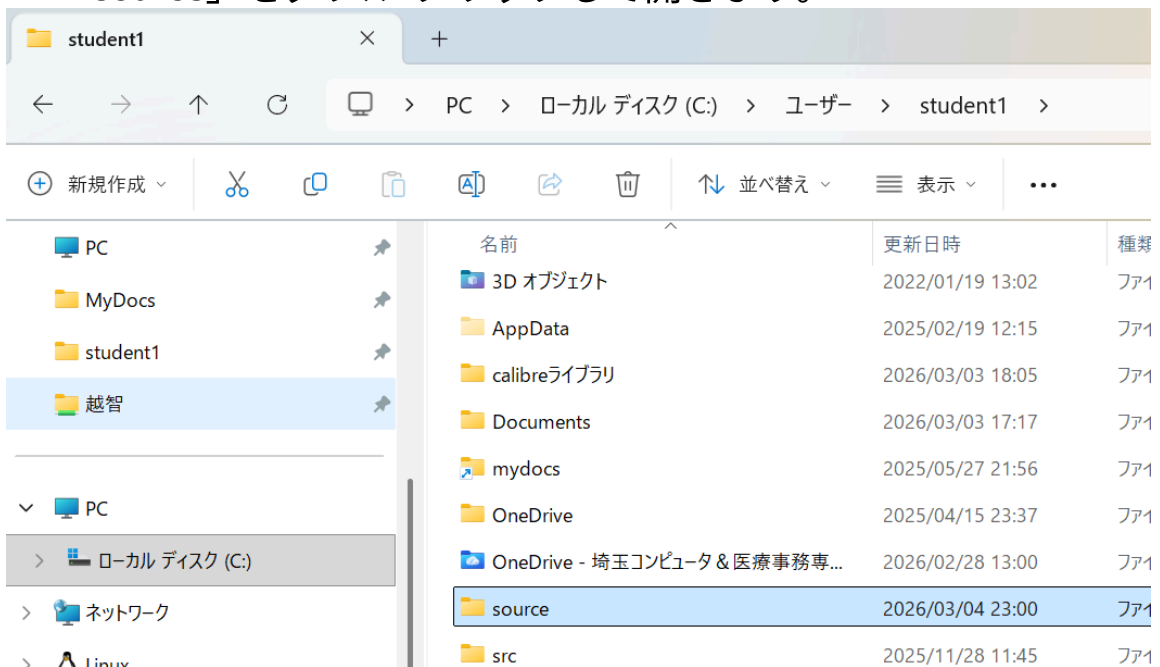
3. ^{みぎ} 右の「ユーザー」をダブルクリックして、^{ひら} 開きます。



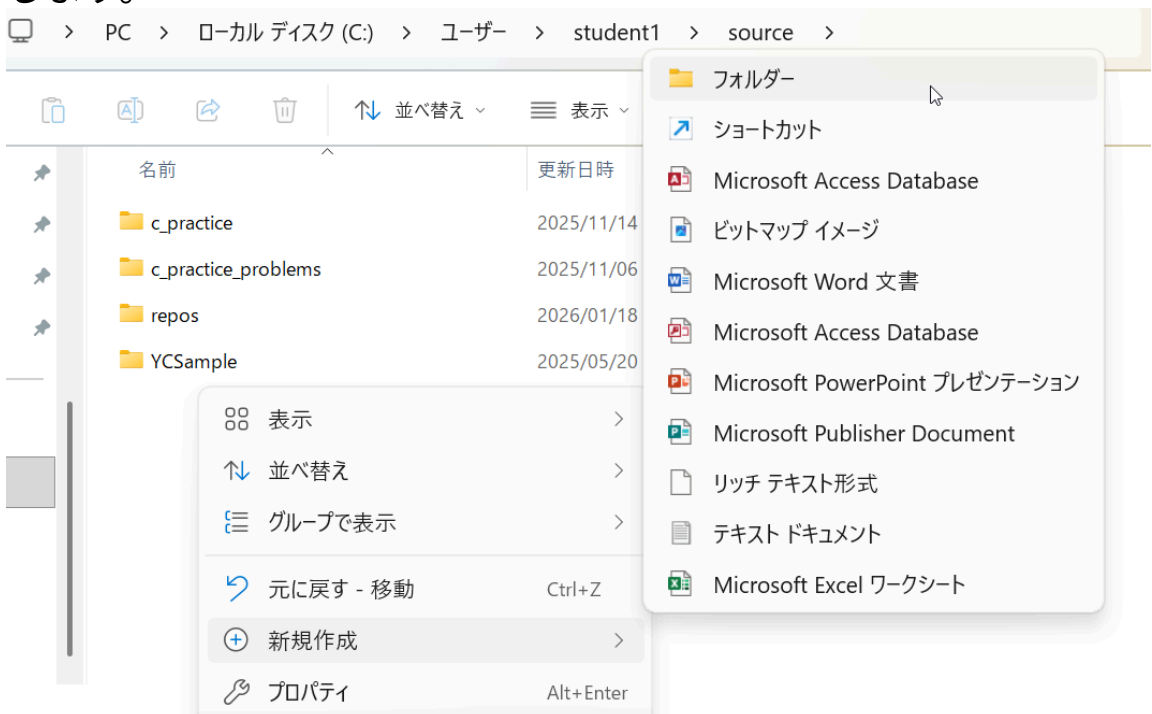
4. 「student1」をダブルクリックして、^{ひら}開きます。



5. 「source」をダブルクリックして開きます。



6. 右クリックして、「新規作成」 → 「フォルダー」をクリックします。



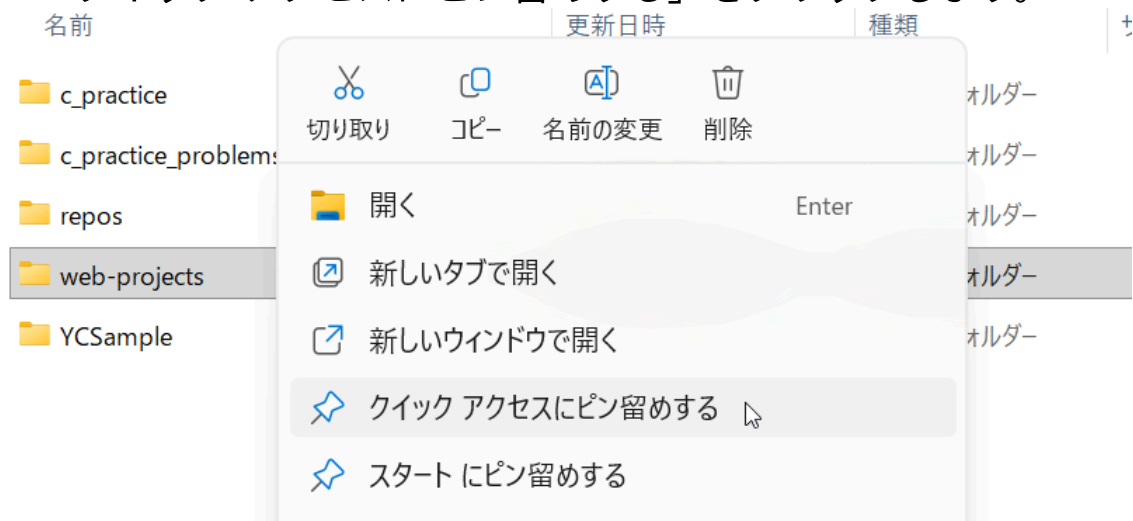
7. ^{あたら}「新しいフォルダー」を「web-projects」に^か書き換^かえます。

名前	更新日時
c_practice	2025/11/14 11:13
c_practice_problems	2025/11/06 21:03
repos	2026/03/04 23:20
YCSample	2025/05/20 22:00
新しいフォルダー	2026/03/04 23:20

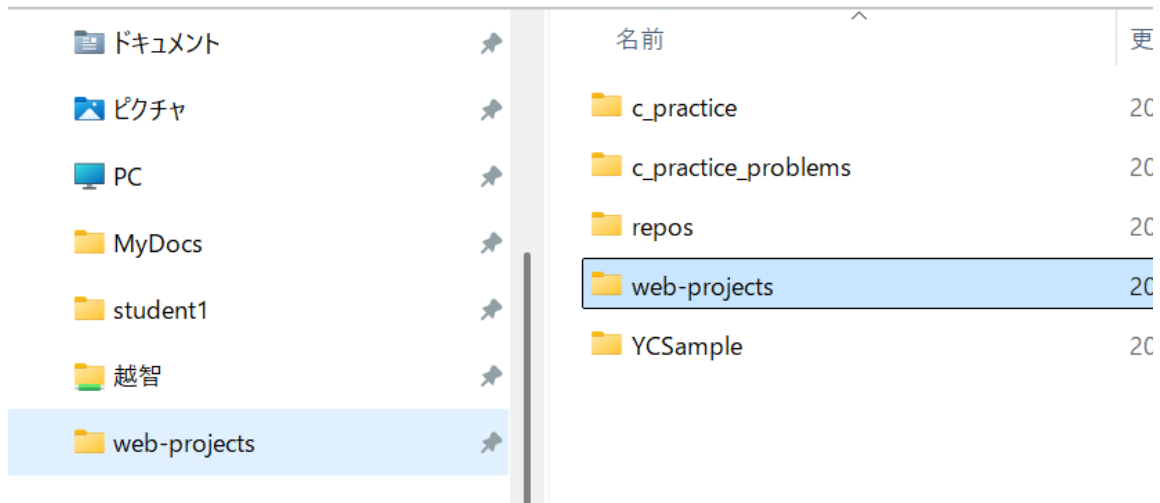
web-projects

(2). ^みすぐに見つけられるように、クイックアクセスにピン留^どめしましょう。

1. 「web-projects」を^{いちど}一度クリックしてから、^{みぎ}右クリックします。
2. 「クイック アクセスにピン留^どめする」をクリックします。

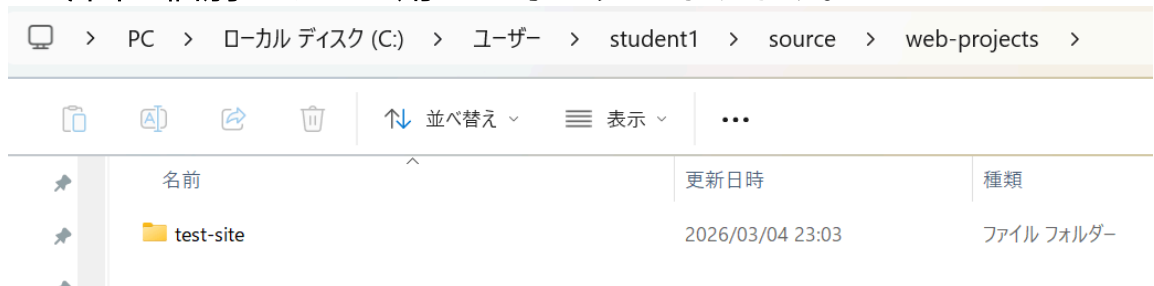


3. 右のクイック アクセスに「web-projects」が表示されるのを確認します。



(3). 「web-projects」の中に個別のサイト用のフォルダを作ります。

1. 5.6.と同じ手順で、「test-site」というフォルダを作ります。これが今回の個別のサイト用のフォルダになります。



場所は、デスクトップやホームフォルダなど、自分ですぐに見つけられる場所にしてください。場所が決まっていると、作業のミスが減り、開発のスピードが速くなります。

次は、ファイルの名前の付け方を説明します。

3. ファイル名とフォルダ名の絶対ルール

名前を付けるときは、世界中のエンジニアが守っている「3つのルール」があります。

1. すべて小文字にする

多くのサーバー（Linuxなど）は、大文字と小文字を厳しく区別します。MyImage.jpg と myimage.jpg は別のファイルだと判断されるため、大文字を使うとエラーの原因になります。

2. 空白（スペース）を入れない

コマンドライン（文字で命令するツール）を使うとき、スペースがあるとコンピュータは「2つの別のファイル」だと勘違いしてしまいます。

3. 言葉を分けるときはハイフン「-」を使う

アンダースコア「_」も使えますが、ハイフンの方がおすすめです。Googleなどの検索エンジンが、ハイフンを「単語の区切り」として正しく認識してくれるからです。

やっていいこと（Do）・やってはいけないこと（Don't）

やっていいこと (Do)	やってはいけないこと (Don't)	理由
test-site	Test Site	大文字とスペースはエラーの原因になります。
my-image.jpg	my image.jpg	スペースがあるとURLが正しく表示されません。

やっていいこと (Do)	やってはいけないこと (Don't)	理由 ^{りゆう}
my-file.html	my_file.html	ハイフンの方が ^{ほう} 検索 ^{けんさく} エンジン ^{この} に好まれます。

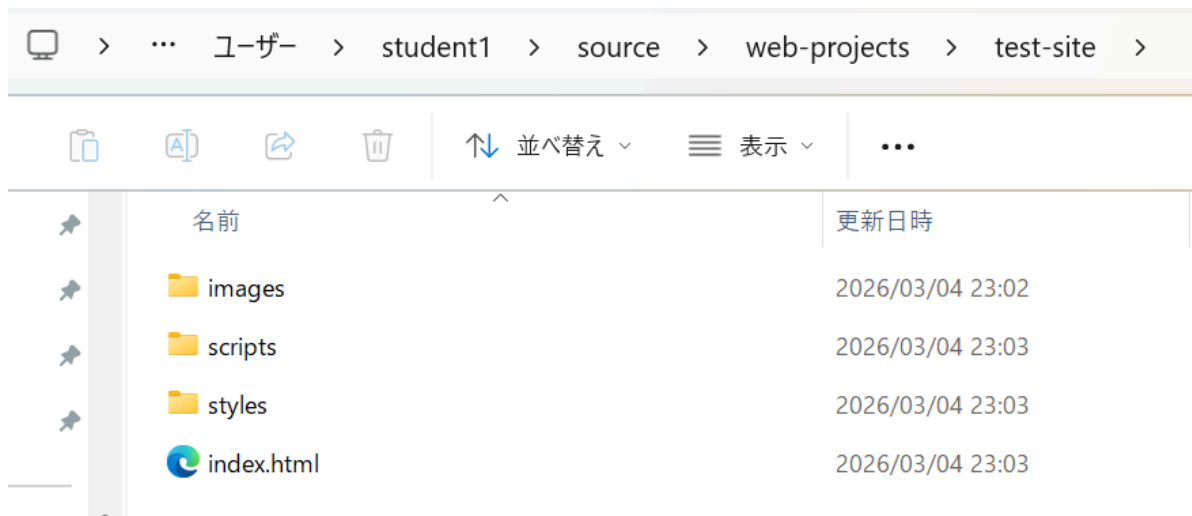
つぎ^{つぎ}は、フォルダの中^{なか}に何^{なに}を入^いれるか説明^{せつめい}します。

4. 標準的なフォルダの形

ウェブ開発では、誰が見ても中身がわかるように、次のような決まった形でフォルダを作ります。

- index.html：ウェブサイトの最初のページです。
- images フォルダ：使う画像やイラストをすべて入れます。
- styles フォルダ：サイトのデザイン（色や形）を決めるCSSファイルを入れます。
- scripts フォルダ：サイトに動きをつけるJavaScriptファイルを入れます。

この「標準的な形」を使うと、他の開発者と一緒に働くときに「どこに何があるか」がすぐに伝わります。



次は、Windowsを使っている人が最初に行うべき設定について説明します。

5. Windowsの設定：拡張子を表示する

Windowsでは、ファイルの種類を表す「.html」や「.jpg」などの拡張子が、既定の設定では隠れています。

しかし、開発では拡張子が重要です。.html ファイルなのか、設定用の .env ファイル（ドットファイル）なのかを正しく判断するために、必ず表示させてください。

拡張子を表示する手順

1. フォルダ（エクスプローラー）を開きます。
2. 上にある [表示] タブ、または [...] メニューから [オプション] を選びます。
3. [表示] タブをクリックします。
4. 詳細設定の中にある [登録されている拡張子は表示しない] のチェックを外します。
5. [OK] をクリックします。

最後に、ファイル同士をつなぐ「ファイルパス」について説明します。

6. ファイルパス：ファイル^{どうし}同士をつなぐ^{みち}道

「ファイルパス」は、あるファイルから別のファイルを探しにいくための「道」のことです。

^か ^{かた}
書き方のパターン

- 同じ場所にあるとき index.html から同じフォルダのファイルを見る場合。例：filename.jpg
- 下のフォルダにあるとき index.html から images フォルダの中のファイルを見る場合。例：images/filename.jpg
- 一つ上のフォルダにあるとき styles フォルダの中のCSSから、外にある images フォルダの中のファイルを見る場合。
例：../images/filename.jpg (..) は「一つ上の階層へ行く」という意味です)

^{たいせつ} ^{ちゅういてん}
大切な注意点

Windowsではフォルダの区切りに「\ (バックスラッシュ)」を使いますが、ウェブ開発では**必ず「/ (スラッシュ)」**を使ってください。\\ を使うと、自分のパソコンでは動いても、インターネットに公開した瞬間にウェブサイトが壊れてしまいます。

まとめ

- 整理整頓：web-projects フォルダを作り、中身をミラー構造で整理しましょう。
- 名前のルール：小文字、スペースなし、ハイフンを使いましょう。
- パスの指定：ウェブでは必ず「/（スラッシュ）」を使いましょう。

ルールを守ってフォルダを作れば、あとの作業がとても楽になります。一步步、楽しくウェブサイトを作っていきましょう。応援しています！

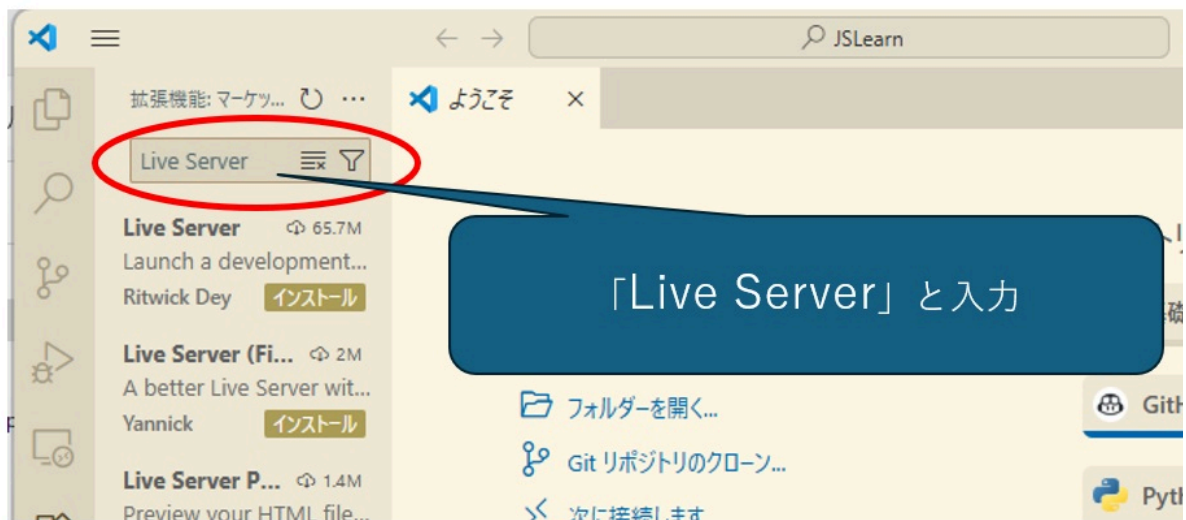
Visual Studio Code の^{じゅんび}準備

Live Server^{かくちょうきのう}拡張機能^{いんすとーる}のインストール

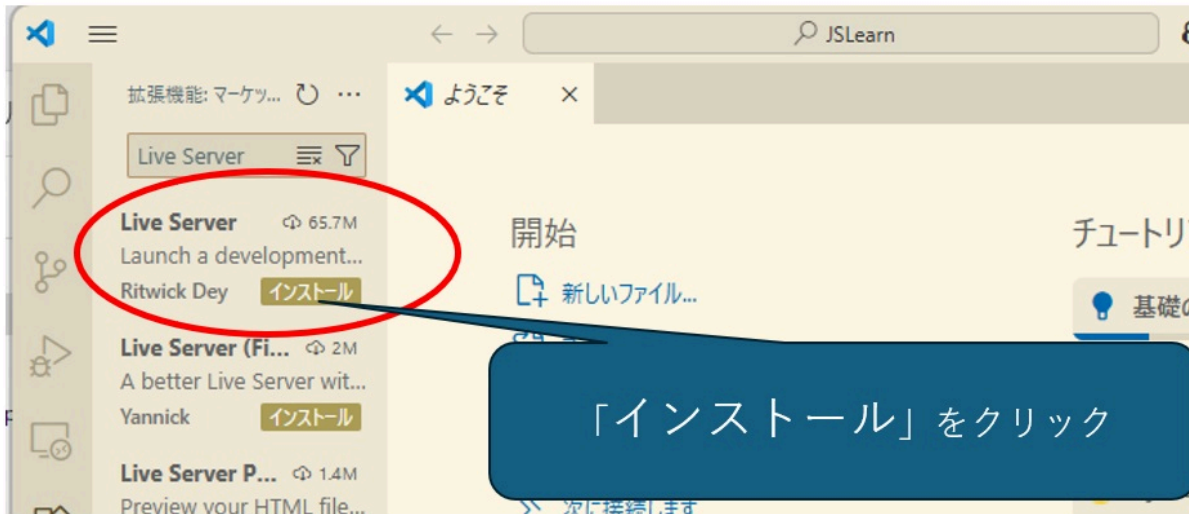
- ^{かくちょうきのう}
1. 拡張機能アイコンをクリックします



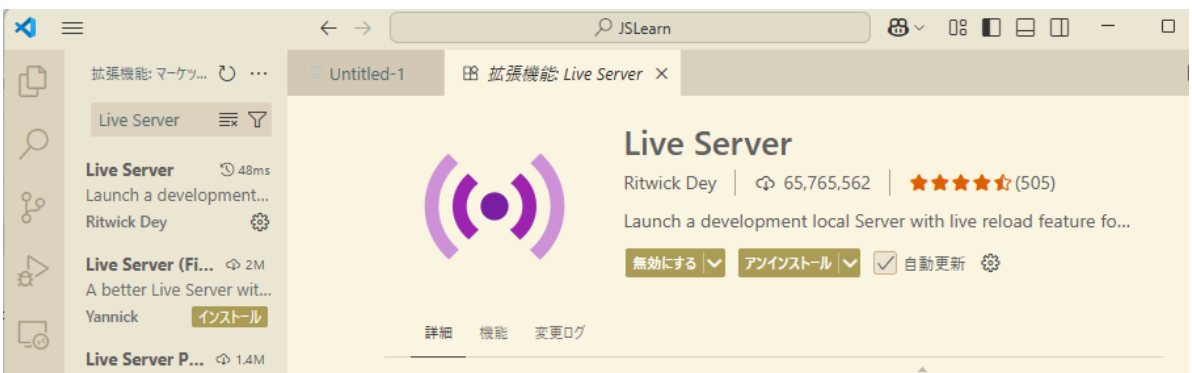
- ^{けんさく}
2. 検索ウィンドウに"Live Server"^{にゅうりよく}と入力します



3. "インストール"をクリックします



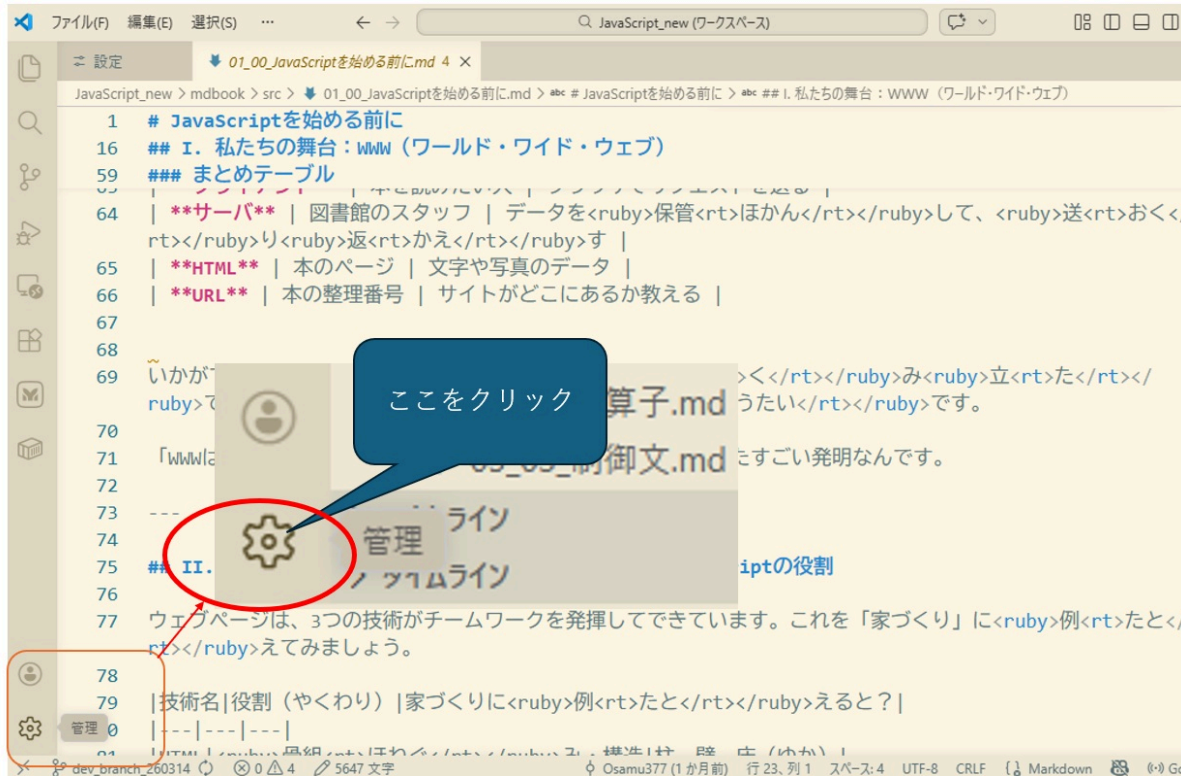
4. インストールされたことを^{かくにん}確認します



げんごせってい 言語設定

ひだりしたすみ はぐるま

1. 左下隅の歯車アイコンをクリック



せってい

1. 「設定」をクリック



けんさくまど にゅうりょく
1. 検索窓に「emmet variables」と入力



こうもく してい
1. 項目「lang」に「ja」を指定して「OK」をクリック

Emmet: Variables
Emmet のスニペットで使用される変数。

項目	値
lang	ja

OK キャンセル

① 項目「lang」を選び、値「ja」を入力

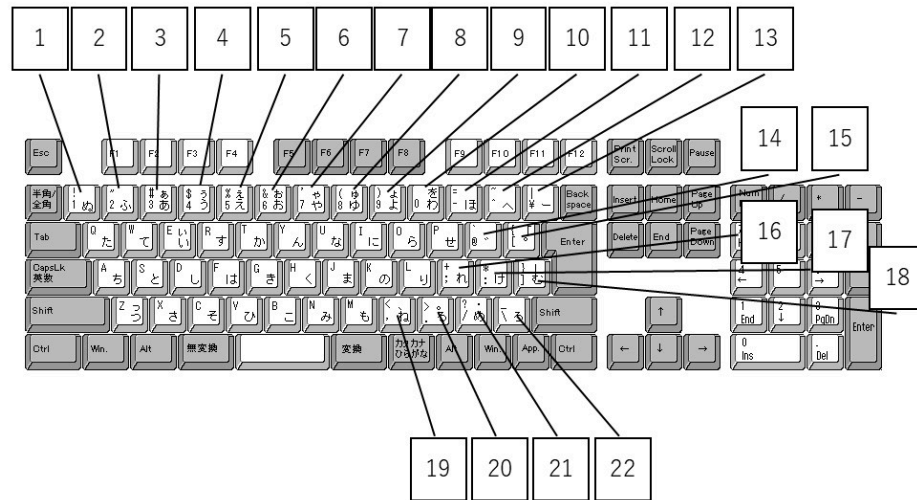
② 「OK」をクリック

はたらき キーの働き

キー	はたらき 働き
→	みぎ 右へ
←	ひだり 左へ
↓	した 下へ
↑	うえ 上へ
Shift+→	せんたくはんい みぎ ひろげる 選択範囲を右へ広げる
Shift+←	せんたくはんい ひだり ひろげる 選択範囲を左へ広げる
Shift+↓	せんたくはんい した ひろげる 選択範囲を下へ広げる
Shift+↑	せんたくはんい うえ ひろげる 選択範囲を上へ広げる
Ctrl+C	コピー
Ctrl+X	きりとり 切り取り
Ctrl+V	はりつけ 貼り付け
Ctrl+F	けんさく 検索
Ctrl+H	ちかん 置換
Ctrl+N	しんき 新規ファイル
Ctrl+O	ひらく ファイルを開く
Ctrl+S	ほぞん 保存
Ctrl+Shift+S	なまえ つけてほぞん 名前を付けて保存

きごう 記号について

にほんご じょう きごう 日本語キーボード上の記号



図のうえ ばんごう 番号	ふつう お 普通に押す	いっしょ お Shiftと一緒に押す
1	1	！ エクスクラメーション マーク / びっくりマーク
2	2	" ダブル クォート
3	3	# 井桁 / シャープ
4	4	\$ ドル
5	5	% パーセント
6	6	& アンド / アンパサン ド
7	7	' シングル クォート
8	8	(かっこひら 開く

図 ^{うえ} の上の ばんごう 番号	ふつう お 普通に押す	いっしょ お Shiftと一緒に押す
9	9	) かっこ閉じる ^と
10	0	(なし)
11	- ハイフン / マイナス	= イコール
12	^ ハット	~ チルダ
13	\ バックスラッシュ ユ / ^{えん} 円マーク	パイプ
14	@ アットマーク	` バッククォート
15	[^{おお} 大 ^{ひら} かっこ開く / ^{かく} 角 ^{ひら} かっこ開く	{ ^{なか} 中 ^{ひら} かっこ開く / ^{なみ} 波 ^{ひら} かっこ開く
16	; セミコロン	+ プラス
17	: コロン	* アスタリスク
18	] ^{おお} 大 ^と かっこ閉じる / ^{かく} 角 ^と かっこ閉じる	} ^{なか} 中 ^と かっこ閉じる / ^{なみ} 波 ^と かっこ閉じる
19	, カンマ / コンマ	< ^{しょう} 小なり
20	. ピリオド / ドット	> ^{だい} 大なり
21	/ スラッシュ	? クエッション マーク / はてなマーク
22	\ バックスラッシュ ユ / ^{えん} 円マーク	_ アンダースコア / アンダーバー

にほんご 日本語のかっこの種類と呼び方

しゅるい よ かた 種類と呼び方

かっこの しゅるい 種類	にほんご よ かた 日本語の呼び方	えいご よ かた 英語の呼び方
()	しょう まる 小 カッコ / 丸 カッコ / カッコ	parenthesis パーレンセ シス paren パーレン round brackets
[]	おお かく 大 カッコ / 角 カッコ	bracket ブラケット square bracket
{ }	なか なみ 中 カッコ / 波 カッコ	brace ブレース curly brace / curly bracket
< >	やま 山 カッコ しょう だい 小 なり / 大 なり	angle bracket less-than sign/greater- than sign